

Algoritmi—Modulo algoritmi per bioinformatica

Introduzione—Capitolo 1 CLRS

Roberto Posenato

versione 2026–3, 04/03/2026

- 1 Introduzione agli algoritmi e ai problemi computazionali
 - Definizione di algoritmo
 - Definizione di algoritmo e problema computazionale
- 2 Esercizi

*“Tell me and I forget, teach me and I remember,
involve me and I learn.”*

Benjamin Franklin

*“A student who achieves knowledge through free investigation
and spontaneous effort will be able to retain that knowledge
and will have acquired a methodology that can serve for a li-
fetime.”*

Jean Piaget

Definizione di algoritmo

Definizione da dizionario

“A procedure for solving a mathematical problem (as of finding the greatest common divisor) in a finite number of steps that frequently involves repetition of an operation.”



m-w.com

Definizione da esperti

“An algorithm is a finite, definite, effective procedure, with some input and some output.”

Donald Knuth—The Art of Computer Programming

Definizione di algoritmo e problema computazionale

Definizione dal libro di testo CLRS

Un **algoritmo** è una procedura di calcolo ben definita che prende un insieme di valori come **input** e genera, in un tempo finito, un valore, o un insieme di valori, come **output**.

Problema computazionale

Un problema computazionale descrive in termini generali la relazione input/output desiderata e che può essere determinata da un algoritmo.

PROBLEMA ORDINAMENTO

INPUT: Sequenza finita di elementi $\langle a_1, \dots, a_n \rangle$ confrontabili con l'operatore $\leq$.

OUTPUT: Una permutazione $\langle a'_1, \dots, a'_n \rangle$ dell'input tale che $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Algoritmo di ordinamento

Procedura di calcolo che risolve il problema dell'ordinamento.

In altre parole, una procedura di calcolo che prende in input una sequenza $\langle a_1, \dots, a_n \rangle$ e produce in output la sequenza ordinata $\langle a'_1, \dots, a'_n \rangle$.

Problemi computazionali e algoritmi

Istanza di un problema

L'istanza di un problema computazionale è formata dall'input che soddisfa tutti i vincoli imposti nella definizione del problema.

Algoritmo corretto

Un algoritmo si dice **corretto** se, per **ogni istanza** del problema, termina con l'output che soddisfa la relazione rappresentata dal problema.

Si dice anche che l'algoritmo **risolve** il problema computazionale dato.

Esempio 1

Data l'istanza $\langle 4, 3, 10, 8, -10, 0, 1 \rangle$ del problema dell'ORDINAMENTO, un algoritmo di ordinamento (corretto) deve restituire come output $\langle -10, 0, 1, 3, 4, 8, 10 \rangle$.

Altro esempio di problema computazionale con possibili output diversi.

PROBLEMA RICERCA IN UNA SEQUENZA

INPUT: Una sequenza $\langle a_1, \dots, a_n \rangle$ e un elemento x .

OUTPUT: Un intero i se $a_i == x$ per un qualche indice i ; messaggio “Non trovato” altrimenti.

Algoritmo di ricerca

Procedura che prende in input $\langle a_1, \dots, a_n \rangle$ ed x , e restituisce in output

- i se $a_i == x$ per un qualche indice i ;
- “Non trovato” altrimenti.

Problemi computazionali e algoritmi

- 1 Dato un problema computazionale, non sempre è possibile trovare un algoritmo che lo risolve.

Esempio 2

Dire se un programma dato in input termina per ogni suo possibile input.

- 2 Non tutti i problemi risolti da algoritmi hanno un gruppo di soluzioni possibili facilmente determinabile.

Esempio 3

Problema della trasformata discreta di Fourier di campioni di segnali.

- 3 Dato un problema computazionale, possono esistere diversi algoritmi che lo risolvono.

Esempio 4

Ordinamento di sequenze di valori confrontabili.

Algoritmi efficienti

Quando un problema computazionale ammette più algoritmi risolutori, è interessante determinare l'/gli algoritmo/i più **efficiente/i**.

- **Efficienza** = uso di *poche risorse*;
- **Risorse** = **Tempo** e **Spazio** richiesti dall'algoritmo per produrre l'output.

Nota!

- ❶ Come misuriamo le risorse usate da un algoritmo? (**analisi degli algoritmi**)
- ❷ Come costruiamo algoritmi che usano *poche* risorse? (**tecniche di progetto**)

Esempio di problema computazionale, suoi algoritmi risolutori e analisi degli algoritmi

Calcolo numero di Fibonacci

PROBLEMA FIBONACCI

INPUT: Un intero non negativo n

OUTPUT: Il valore F_n , n -esimo valore della successione di Fibonacci.

Nota!

La successione di Fibonacci è definita ricorsivamente come:

$$F_n = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F_{n-1} + F_{n-2} & \text{if } n > 1 \end{cases}$$

n	0	1	2	3	4	5	6	7	8	9	10	11	12	13	...
F_n	0	1	1	2	3	5	8	13	21	34	55	89	144	233	...

Primo algoritmo per prob. FIBONACCI: FIBONACCIR

Algoritmo: FIBONACCIR

Input: Intero non negativo n

Output: Il valore F_n

```
1: if  $n == 0$  then return 0
2: if  $n == 1$  then return 1
3: return FIBONACCIR( $n - 1$ ) + FIBONACCIR( $n - 2$ )
```

❶ L'algoritmo è *corretto*? **Sì!**

- Per ogni input valido, termina? **Sì, assumendo che input sia sempre un intero $n \geq 0$**
- Se sì, calcola il valore giusto? **Sì, perché l'algoritmo codifica la definizione ricorsiva.**

❷ Quanto **tempo** richiede per terminare?

❸ Possiamo fare meglio?

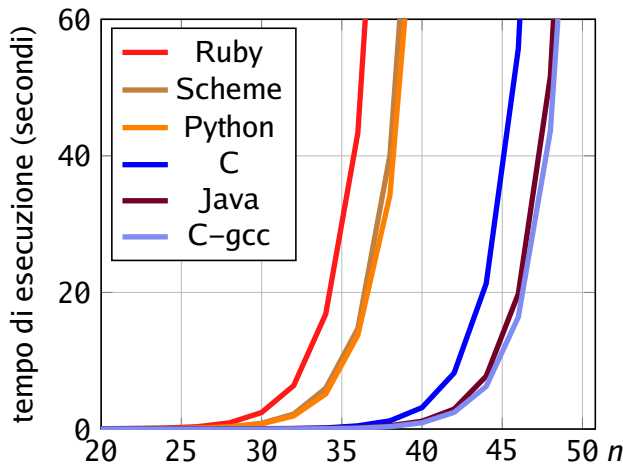
Nota!

Le convenzioni usate per il pseudocodice sono a pag. 19 del [CLRS].

Primo algoritmo per prob. FIBONACCI: FIBONACCIR

Quanto tempo richiede per terminare?

Un primo tentativo, ingenuo, di valutazione...



Grazie al Prof. Carzaniga per i dati di questa tabella resi disponibili a <https://www.inf.usi.ch/carzaniga/edu/material>

Primo algoritmo per prob. FIBONACCI: FIBONACCIR

Quanto tempo richiede per terminare?

- Le differenti implementazioni hanno tempi di esecuzioni diversi;
- Tutte però presentano un crescita esponenziale del tempo di esecuzione;
- Quindi, tutte presentano un limite pratico oltre il quale il programma non è in grado di calcolare il valore in un tempo finito.
- Nessuna ci permette di dare una stima del tempo per qualsiasi valore di n .

Primo algoritmo per prob. FIBONACCI: FIBONACCIR

Quanto tempo richiede per terminare?

- C'è bisogno di una caratterizzazione matematica del tempo di esecuzione che sia indipendente dalla particolare implementazione.
- Questa caratterizzazione è detta **complessità computazionale** dell'algoritmo.
- **In prima approssimazione**: si denota con $T(n)$ il numero di **istruzioni elementari** che si devono eseguire quando l'input è "caratterizzato" da n :
 - In generale le istruzioni elementari sono: *assegnamenti e operazioni aritmetiche e logiche*.
 - Un ciclo **for** costa
 $m \cdot (\text{numero istruzioni elementari dentro il ciclo}) + (m + 1) \cdot c_{for}$,
dove m è il numero di cicli eseguiti e $c_{for} \approx 2$ è il costo del controllo del **for**.
 - Un **return** e una chiamata a funzione costa 1.

Primo algoritmo per prob. FIBONACCI: FIBONACCIR

Quanto *tempo* richiede per terminare?

Algoritmo: FIBONACCIR

Input: Intero non negativo n

Output: Il valore F_n

```
1: if  $n == 0$  then return 0  
2: if  $n == 1$  then return 1  
3: return FIBONACCIR( $n - 1$ ) + FIBONACCIR( $n - 2$ )
```

Si determina che:

$$T(0) = 2$$

$$T(1) = 3;$$

$$T(n) = T(n - 1) + T(n - 2) + 8$$

$$\Rightarrow T(n) > F_n$$

Il +8 perché si ci sono i due **if** riga 1 e 2 (2 operazioni), poi si devono calcolare i parametri (2 operazioni), fare le chiamate (2 operazioni), fare la somma (1 operazione) e il **return** (1 operazione).

Primo algoritmo per prob. FIBONACCI: FIBONACCIR

Complessità dell'algoritmo

In prima approssimazione:

- Si studia come cresce $T(n)$ con l'aumentare di n (che è la **dimensione** dell'input)

$$T(n) > F_n = F_{n-1} + F_{n-2}$$

- Per $n \geq 7$, $F_n > F_{n-1} > F_{n-2} > F_{n-3} > \dots$

$$F_n > 2F_{n-2} > 2(2F_{n-4}) > 2(2(2F_{n-6})) > \dots > 2^{\frac{n}{2}}$$

- Quindi

$$T(n) > (\sqrt{2})^n \approx (1.4)^n$$

- $T(n)$ **cresce esponenzialmente** con n
- Inciso: una stima più precisa è $T(n) \geq \left(\frac{\phi^n - (-\phi)^{-n}}{\sqrt{5}} \right)$, dove $\phi = \frac{1+\sqrt{5}}{2} \approx 1,62$ (Formula di Binet).
- Si può scrivere un algoritmo che richieda minor tempo computazionale?

Secondo algoritmo per prob. FIBONACCI: FIBONACCI

- Si deve evitare di ricalcolare i valori ricorsivamente;
- Si calcolano i valori sommando i valori dal più piccolo al più grande!

Algoritmo: FIBONACCI

Input: Intero positivo n

Output: Il valore F_n

```
1: if  $n == 0$  then return 0
2: if  $n == 1$  then return 1
3:  $pprev = 0$ 
4:  $prev = 1$ 
5: foreach  $i = 2$  to  $n$  do
6:    $f = prev + pprev$ 
7:    $pprev = prev$ 
8:    $prev = f$ 
9: return  $f$ 
```

Secondo algoritmo per prob. FIBONACCI: FIBONACCII

Complessità dell'algoritmo

Algoritmo: FIBONACCII

Input: Intero positivo n

Output: Il valore F_n

```
1: if  $n == 0$  then return 0
2: if  $n == 1$  then return 1
3:  $pprev = 0$ 
4:  $prev = 1$ 
5: foreach  $i = 2$  to  $n$  do
6:    $f = prev + pprev$ 
7:    $pprev = prev$ 
8:    $prev = f$ 
9: return  $f$ 
```

Calcolo delle complessità
computazionale:

$$T(0) = 2$$

$$T(1) = 3$$

$$T(n) = 4 + 2n + 4(n - 1) + 1 = 6n + 1$$

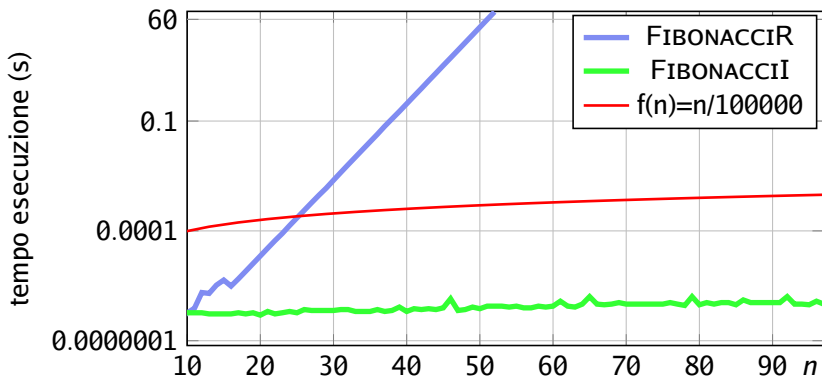
Il controllo del **for** (incremento e confronto)
costa $2n$ in totale.

La complessità (computazionale) di FIBONACCII(n) è **lineare** in n .

Confronto complessità degli algoritmi per FIBONACCI

Tempi di computazione FIBONACCIR e FIBONACCII entrambi implementati in Java ed eseguiti su un PowerBook M1. Per valori $n > 97$, non è possibile rappresentare il valore F_n con 64 bit.

L'asse Y è in scala logaritmica per mostrare la complessità lineare di FIBONACCII.



Importanza dell'efficienza algoritmica

L'efficienza di un algoritmo ha maggior impatto dell'efficienza dell'hardware e dell'implementazione.

- Algoritmo di ordinamento **insertion sort** ha complessità
 $c_1 n^2 = (c_1 n) \cdot n$
- Algoritmo di ordinamento **merge sort** ha complessità
 $c_2 n \lg n = (c_2 \lg n) \cdot n$
- Per quasi tutte le varianti di questi due algoritmi, vale $c_1 < c_2^*$
- Il computer *veloce* **A** esegue un'implementazione efficiente di **insertion sort** dove $c_1 = 2$
- Il computer *lento* **B** esegue un'implementazione di **merge sort** dove $c_2 = 50$

*I loro valori esatti non sono fondamentali come vedremo in seguito.

Importanza dell'efficienza algoritmica

- Entrambi devono ordinare 10 milioni di interi ($10 \cdot 10^6 = 10E6 = 1E7$).
- *A* esegue 10E9 istruzioni al secondo (≈ 10 gigaFLOPS).
- *B* esegue 10E6 istruzioni al secondo ($\approx 1/100$ gigaFLOPS).
- *A* ordina l'input in tempo $\frac{2 \cdot (10^7)^2}{10^{10}} = 2 \cdot 10^4 = 2E4$ s (> 5.5 ore).
- *B* ordina l'input in tempo $\frac{50 \cdot (10^7) \lg 10^7}{10^7} \approx 1163 \approx 1E3$ s (< 20 m).
- Nonostante il 1000 volte più lento *B* esegua un'implementazione scarsa, è 17 volte più veloce di *A* nel dare l'output.

In questa lezione si è visto:

- Possono esistere diversi algoritmi per risolvere un problema computazionale;
- Per stimare il comportamento di un programma possiamo contare il numero di operazioni rispetto alla “dimensione” dell’input;
- Tecniche algoritmiche possono migliorare di molto il comportamento dei programmi;

Esempi di problemi computazionali da risolvere in aula

Two Sum

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

You can return the answer in any order.

Esempi di problemi computazionali da risolvere in aula

Reverse Integer

Given a signed 32-bit integer x , return x with its digits reversed. If reversing x causes the value to go outside the signed 32-bit integer range $[-2^{31}, 2^{31} - 1]$, then return 0.

Assume the environment does not allow you to store 64-bit integers (signed or unsigned).

Esempi di problemi computazionali da risolvere in aula

Container With Most Water

You are given an integer array `height` of length n . There are n vertical lines drawn such that the two endpoints of the i th line are $(i, 0)$ and $(i, \text{height}[i])$.

Find two lines that together with the x-axis form a container, such that the container contains the most water.

Return the maximum amount of water a container can store.

Notice that you may not slant the container.

Esempi di problemi computazionali da risolvere in aula

CLongest Common Prefix

Write a function to find the longest common prefix string amongst an array of strings.

If there is no common prefix, return an empty string `""`.

- Studiare cap. 1 [CLRS].
- Risolvere gli esercizi 1.2–2, 1.2–3 [CLRS].
- Risolvere il problema 1–1 [CLRS].